Phase 6 — Bull/Bear/Judge-Debatte

trading-bot-lizard-branch · phase6_debate/ · erster Mehr-Agenten-AI-Pfad & teuerster Pfad im

code-complete
live-verifiziert · main 49cda8a

Bauprinzip: **so wenig AI wie möglich, so viel AI wie nötig.** AI nur: Bull/Bear-Argumente + Judge-Synthese — alles andere ist Code.

Die Naht im Executor — debate_runner zwischen Gate 2 und Gate 3 (Decision A · default No-Op → P5 AI-frei)

P4 Research
Gate 2 → candidate
(SEED: epic+dir)

→

phase6_debate · debate_runner(candidate) → DebateOutcome
Judge ist die ECHTE Richtungsquelle — darf den Seed FLIPpen (Decision B).
Bekommt NIE Broker/VETO/Gate → kann strukturell keinen VETO umgehen (Decision J).
Jeder Fehler / Cap-Breach / Context-Fehler → ABSTAIN (fail-closed).

→

P5 Executor
Gate 3/5 · Sizing
4 VETOs · Order

TRADE → angereicherter (ggf. geflippter) Candidate in-memory weiter · ABSTAIN → NO_TRADE

Im debate_runner

context_builder.py (CODE)
= DIE AI-GRENZE. Liest GENAU 1× frischen Broker-Snapshot:
get_price · get_ohlcv · get_market_info
→ frozen DebateContext (Seed + IG-Echtzeit-Spread + recent_bars).

audit.py → debate_log.json
gitignored · DB bleibt read-only (bis P7).
Schema vorwärtskompatibel → P9-Tabellen.

Runden-Loop (max_rounds=5 · Early-Stop)

BULL
Haiku 4.5
pro LONG

→

BEAR
Haiku 4.5
pro SHORT

→

JUDGE
Sonnet 4.6
decision+dir

Judge conclude=true → Early-Stop. Budget-Caps nach JEDEM Turn:
calls ≤18 · in ≤120k · out ≤12k · wall-clock ≤120s → Breach = ABSTAIN

confidence ist ADVISORY und gatet NICHTS (Decision H, ECE~0.1).
Nur Judge-decision ∈ {TRADE, ABSTAIN} entscheidet.

Transkript-Kompression (Code, kein Extra-Call) bei 65% Token-Budget.

GOTCHA — Caching-Realität (Live-Lauf 2026-06-16, Code = Source of Truth)

Haiku-4.5 Mindest-Cache-Präfix = **4096 Tokens** ; Sonnet-4.6 = 2048. Bull/Bear (Haiku, ~850–1040 Input-Tokens) liegen WEIT darunter → die billigen Agenten cachen **nie** (cache_creation=0). Nur der Judge (Sonnet) schreibt Cache. Ein Cache- **READ** braucht denselben Präfix ≥2× → entsteht erst bei **rounds_used ≥ 2** . Ein 1-Runden-Early-Stop liest nichts → cache_read=0 ist KORREKT (war das einzige rote Live-Assertion, jetzt gefixt).

Konsequenz für P7: „Caching“ ist NICHT der große Hebel, den der Plan annahm. Mehr Hebel: kurze stabile Judge-Prompts + Batch-API (50%) für Lesson-Extra

Module — TDD, alle fertig · pytest: P6 70 / P5 135 / P4 88 grün

- models.py — Contract-Typen (frozen)
- context_builder.py — AI-Grenze
- timeutil.py — ISO-UTC-Seam
- prompts.py — Bull/Bear/Judge (DE)
- llm_caching.py — cache_control + Meter
- budget.py — Caps, fail-closed
- debate.py — Orchestrator (Kern)
- audit.py — debate_log.json
- credentials.py — keyring-Shim
- scripts/wiring.py — build_executor
- scripts/smoke_test.py — dry
- scripts/live_test.py — Operator
- P5 executor.py — Naht (Decision A)
- P5 test_executor.py — +5 Regression

additive P4-Edits (Default unverändert):
llm_client.cache_system · token_meter +2

Gelockte Entscheidungen

A platzierung: runner @ Gate 2→3
B Judge darf SEED flippen
C Bull/Bear=Haiku · Judge=Sonnet
D single-shot, Transkript→user
E early-stop + harte Caps
F reuse research.* (dok. Ausnahme)
G token_meter +cache-Felder
H confidence = advisory, gatet nichts
I JSON-Audit, kein SQLite (→P9)
J fail-closed ABSTAIN · K Council→P9

live-verifiziert ≠ profit-validiert

Der Live-Lauf bewies den GANZEN Pfad end-to-end (3 echte Calls → fail-closed ABSTAIN → NO_TRADE → debate_log.json) — aber NUR EIN ABSTAIN-Zyklus, KEINE Order durch die Debatte, KEIN gemessener Edge. Ob die zweite AI-Schicht den P4-Pick verbessert oder nur teurer macht, ist offen → Phase 7.

Diagramm-Konvention: GitHub-Dark · monospace · grün = fertig · farbige Kreise statt Emoji · 960er viewBox. Quelle der Wahrheit bleibt der Code.

Nächste Phase: **Phase 7 — Reward/Brain** (ECC, outcome-anchored confidence: f(win_rate, expectancy, sample_size), N_min=10).
P7 = erste Phase, die in SQLite SCHREIBT. Substrat liegt bereit: debate_log.json + advisory confidence.